

FBCBot – System Design Document

1. Architecture Strategy

- Start with single-church deployment
- Design database to support multi-church scaling
- Include church_id in all major tables

2. Data Privacy & Access Control

- Implement Role-Based Access Control (Admin, Prayer Team, Member)
- Restrict access to sensitive data like prayer requests
- Provide anonymous submission options

3. Bot Interaction Model

- Use button-based navigation instead of commands
- Design simple and intuitive Telegram UI
- Ensure flows are easy for non-technical users

4. State Management

- Use database-backed state tracking
- Track user flow steps (e.g., prayer request, quiz)
- Avoid in-memory state storage

5. Data Modeling Strategy

- Use structured schema for core data
- Use JSON fields for flexible data like requests and quiz answers

6. Notifications Strategy

- Support broadcast, targeted, and scheduled messages

- Start with direct messaging
- Scale later using background job queues

7. Admin Dashboard

- Build a web-based admin panel using Reflex
- Allow management of sermons, announcements, and requests

8. File Storage

- Use cloud storage (AWS S3, Cloudinary, or similar)
- Avoid storing files directly in the database

9. Deployment

- Start with simple cloud hosting (Render, Railway)
- Ensure easy scalability

10. Scalability

- Keep backend stateless
- Prepare for horizontal scaling

11. Logging & Monitoring

- Implement basic logging
- Add monitoring tools later (e.g., Sentry)

12. Security

- Limit access to sensitive data
- Avoid exposing personal information
- Secure admin endpoints

13. Feature Toggles

- Allow churches to enable/disable features
- Store settings per church

14. WhatsApp Expansion

- Abstract business logic from bot interface
- Reuse backend for Telegram and WhatsApp

15. Data Retention Policy

- Define retention period for prayer requests and sensitive data
- Auto-delete after defined duration

16. System Overview

Telegram Bot handles user interaction, Backend (Reflex) manages logic and APIs, Database stores application data, Admin Dashboard controls content, and Cloud Storage manages media files.